第9章: スタイリングとUI（NativeWind）

> アプリの機能は完成しました。この章では「見た目」をより効率よく整える方法を学びます。本書では **NativeWind**（Tailwind CSSの書き方をReact Nativeで使えるライブラリ）を導入します。まずスタイリング手法の選択肢を比較し、NativeWindを導入して、これまで作った画面をより少ないコードで書き直します。

---

1. スタイリング手法の選択肢を比較する

第4章では `StyleSheet` を使いました。React Nativeのスタイリングには他にも選択肢があります。「どんな場合にどれを選ぶべきか」を解説します。

1.1 比較表

手法	記述スタイル	記述量	学習コスト	Web版との共通性	向いている人
StyleSheet（RN標準）	オブジェクトで定義	多い	低（追加不要）	△	追加ライブラリを増やしたくない人
NativeWind（本書採用）	クラス名で指定	少ない	低（Tailwind経験者は即）	◎	Web/TailwindとUIを共通化したい人
Tamagui	専用コンポーネント	中	中〜高	△	高性能・高機能を求める人
UIキット（gluestack / React Native Paper等）	完成部品を使う	少ない	中	△	デザインを自分で考えず素早く作りたい人

1.2 それぞれの解説

◆ StyleSheet（RN標準）

第4章で使った方法。`StyleSheet.create({ ... })` でスタイルを定義し、名前で参照します。

- **長所**: 追加ライブラリ不要／React Nativeの基本そのもの／どんな教材でも通用する
- **短所**: 記述量が多い／小さな調整でもスタイル定義が増えていく

◆ NativeWind（本書の採用）

Tailwind CSS という「短いクラス名で見た目を指定する」仕組みを、React Nativeで使えるようにしたもの。
`className="text-lg font-bold text-blue-600"` のように書きます。

- **長所**: 記述量が少ない／Web版チュートリアル（Tailwind）と書き方が同じ／デザインの一貫性を保ちやすい
- **短所**: クラス名を覚える必要がある（が、よく使うものは限られる）

◆ Tamagui（タマグイ）

高性能を売りにした高機能スタイリング＋UIライブラリ。アニメーションやテーマ切替に強い。

- **長所**: 高性能／機能豊富
- **短所**: 学習コストが高め／初心者にはオーバースペックなことが多い

◆ UIキット（gluestack-ui / React Native Paper など）

ボタンやカードなどの「完成された部品」が最初から用意されているライブラリ。

- **長所**: デザインを考えなくても整った見た目になる／開発が速い
- **短所**: キット独自の作法を学ぶ／細かいカスタマイズに制約

> **どんな時にどれを選ぶ？（まとめ）** - **Web/Tailwindの経験があり、UIを共通化したい** → **NativeWind（本書）** - **ライブラリを増やさず基本に忠実でいたい** → StyleSheet - **凝ったアニメーションや高性能が必要** → Tamagui - **デザインを自分で作らず最速で形にしたい** → UIキット

---

2. NativeWind のセットアップ

2.1 インストール

第1章で作った `my-books-app` フォルダで、次を順に実行します。

コピー

```
npx expo install nativewind tailwindcss react-native-reanimated react-native-safe-area-context
# nativewind                    : 本体（TailwindをRNで使えるようにする）
# tailwindcss                   : Tailwindのクラス定義の土台
# react-native-reanimated       : NativeWindが内部で利用するアニメーション部品
# react-native-safe-area-context: 安全領域の部品（第4章で登場。NativeWindと相性よく使う）
```

2.2 Tailwind設定ファイルを作る

次のコマンドで、Tailwindの設定ファイルのひな形を生成します。

コピー

```
npx tailwindcss init
# tailwindcss init : tailwind.config.js という設定ファイルを生成するコマンド
```

生成された `tailwind.config.js` を次のように書き換えます。

> **▼ このコードがやること（先に日本語で）**：NativeWind（Tailwind）に「**クラス名をどのファイルから探すか**」と「**NativeWind用の設定一式を読み込む**」ことを教える設定ファイルです。一番大事なのは `content` ——ここに書いたファイルの中で実際に使われているクラスだけがスタイルとして取り込まれます。書き換える箇所は数行だけで、残りはほぼ決まり文句（おまじない）です。詳しい意味は各行のコメントを見てください。

コピー

```
// tailwind.config.js – Tailwind/NativeWindの設定

/** @type {import('tailwindcss').Config} */
module.exports = {
  // content : Tailwindのクラスを「どのファイルから探すか」を指定する
  // app配下とcomponents配下の .tsx などを対象にする
  content: ["./app/**/*.{js,jsx,ts,tsx}", "./components/**/*.{js,jsx,ts,tsx}"],
  // presets : NativeWind用の設定一式を読み込む（おまじない）
  presets: [require("nativewind/preset")],
  theme: {
    // ここに独自の色やサイズを追加できる（今は空でOK）
    extend: {},
  },
  plugins: [],
};
```

> `content` **の指定が重要:** Tailwindは「実際に使われているクラスだけ」を最終的に取り込みます。`content` で指定したファイルからクラス名を探すので、**ここにファイルの場所を正しく書かないとスタイルが効きません。** `./app/**/*.{...}` は「appフォルダ以下の全階層（ `**` ）の、指定拡張子のファイル」という意味です。

▼ コードを1つずつ分解して解説

解説1: 設定全体の入れ物（`module.exports`）

コピー

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  // ...（中身は下で順に解説）
};
```

- `module.exports = { ... }` は「**このファイルが外に渡す内容**」を指定する書き方です。`tailwind.config.js` 全体が「1つの設定オブジェクト（`{ }`）」を外に公開しています。
- 1行目の `/** @type ... */` は **コメント**で、エディタに「これはTailwindの設定オブジェクトですよ」と教える目印です。書いておくと入力補完が効きますが、消しても動作は変わりません。
- 設定オブジェクトの中に `content` / `presets` / `theme` / `plugins` という4つの項目を並べていきます。

> **用語:** `module.exports` … JavaScriptのファイル（モジュール）が「自分の中身を他のファイルに渡す」ための仕組み。ここに代入したものが、他の場所から読み込めるようになります。

解説2: クラスを探す場所（`content`）

コピー

```
  content: ["./app/**/*.{js,jsx,ts,tsx}", "./components/**/*.{js,jsx,ts,tsx}"],
```

- `content` は「**`className` をどのファイルから探すか**」のリストです。ここに書いたファイルの中で実際に使われているクラスだけが、最終的なスタイルに取り込まれます。
- `./app/**/*.{js,jsx,ts,tsx}` は「`app` フォルダ以下の**全階層**（`**`）にある、拡張子が `js` / `jsx` / `ts` / `tsx` のファイル」という意味です。`components` フォルダも同様に対象にしています。
- ここを書き忘れたり場所を間違えると、**クラスが見つからずスタイルが何も効かなくなる**ので、最重要の項目です。

> **用語:** グロブ（glob） … `**` や `*` を使ってファイルの場所をまとめて指定する書き方。`**` は「何階層でも下までたどる」、`*` は「任意のファイル名」を表します。

解説3: NativeWind用設定と拡張枠（`presets` / `theme` / `plugins`）

コピー

```
  presets: [require("nativewind/preset")],
  theme: {
    extend: {},
  },
  plugins: [],
```

- `presets` は「**あらかじめ用意された設定一式を読み込む**」項目です。`require("nativewind/preset")` でNativeWind用の設定をまとめて取り込んでいます（ほぼ決まり文句）。
- `theme.extend` は「**独自の色やサイズを追加する**」場所です。今は `{}`（空）でOK。たとえば自社カラーを足したくなったらここに書きます。
- `plugins` は「追加機能（プラグイン）を入れる」リストで、今回は不要なので空配列 `[]` のままにしています。

> **用語:** `require(...)` … 他のファイルやライブラリの中身を読み込む関数。`module.exports` で公開されたものを、こちらで受け取るイメージです。

2.3 グローバルCSSを作る

`app` フォルダに `global.css` を新規作成し、Tailwindの基本指定を書きます。

> **▼ このコードがやること（先に日本語で）:** Tailwindの土台となるスタイルを読み込むための、**3行だけのCSSファイル**を作ります。`@tailwind base / components / utilities` の3行を書くことで、`text-lg` や `bg-white` といった便利クラスがアプリで使えるようになります。これはNativeWindを使うための「お決まりの3行」なので、丸ごとそのまま書けばOKです。

コピー

```
/* app/global.css – Tailwindの基本スタイルを読み込む */
/* Tailwindの土台スタイル */
@tailwind base;
/* コンポーネント系スタイル */
@tailwind components;
/* text-lg などの便利クラス群 */
@tailwind utilities;
```

▼ コードを1つずつ分解して解説

解説1: 3つの `@tailwind` がそろってTailwindが使えるようになる

コピー

```
/* Tailwindの土台スタイル */
@tailwind base;
/* コンポーネント系スタイル */
@tailwind components;
/* text-lg などの便利クラス群 */
@tailwind utilities;
```

- `@tailwind` は「**Tailwindの該当部分のスタイルを、ここに展開してね**」とCSSに指示する書き方です。3行それぞれが別の役割を持っています。
- `base` … 文字や余白などの「**土台となる初期スタイル**」を読み込みます。ブラウザごとの見た目のばらつきをそろえる役目です。
- `components` … ボタンやカードといった「**部品向けのスタイル**」を読み込む枠です。
- `utilities` … `text-lg` や `bg-white` のような「**短いクラス名で1つの見た目を当てる便利クラス群**」を読み込みます。NativeWindで毎回使うのは主にこれです。
- この3行はNativeWindを使うための**お決まり**なので、丸ごとそのまま書けばOKです。

> **用語:** ユーティリティクラス … `p-4`（余白）や `text-blue-600`（文字色）のように「1つの見た目だけを当てる小さなクラス」のこと。これらを並べて組み合わせるのがTailwind/NativeWindの基本スタイルです。

2.4 Babel と Metro の設定

NativeWindを動かすため、2つの設定ファイルを調整します。内部の仕組みなので「こう書く」と覚えれば十分ですが、**それぞれが何をしているのか**を知っておくと、エラー時に対処しやすくなります。

> **そもそも Babel と Metro とは？** あなたが書く `className="text-lg"` のようなコードは、そのままではスマホは理解できません。アプリを動かす前に、**スマホが分かる形へ変換（へんかん）**する道具が必要です。- **Babel（バベル）** … 新しい書き方（TypeScript、JSX、NativeWindのclassName）を、スマホが解釈できる素のJavaScriptへ**翻訳する**道具。- **Metro（メトロ）** … たくさんのファイルや部品を**1つにまとめて**スマホへ届ける道具（「バンドラ＝束ねるもの」と呼びます。第1章で登場）。
>
> NativeWindは「`className` を本物のスタイルへ変換する」仕組みなので、この**BabelとMetroの両方に「NativeWindの変換も通してね」と教える**必要があります。それが下の2ファイルの役割です。

`babel.config.js`（無ければ作成）：

> **▼ このコードがやること（先に日本語で）：** Babel（コードをスマホが分かる形へ翻訳する道具）に「**classNameをスタイルに変える翻訳ルールも使ってね**」と教える設定です。`jsxImportSource: "nativewind"` と `"nativewind/babel"` の2つが、その指定の中心です。仕組みは下のblockquoteで詳しく説明していますが、まずは丸ごとコピーで構いません。詳細は各行のコメントを参照してください。

コピー

```
// babel.config.js – コード変換の設定
module.exports = function (api) {
  // 設定をキャッシュして高速化（おまじない）
  api.cache(true);
  return {
    // presets : 変換ルールの一式。Expo用 + NativeWind用（jsxImportSource）を指定
    presets: [
      ["babel-preset-expo", { jsxImportSource: "nativewind" }],
      "nativewind/babel",
    ],
  };
};
```

▼ コードを1つずつ分解して解説

解説1: 設定を関数で返す形（`module.exports = function (api)`）

コピー

```
module.exports = function (api) {
  // 設定をキャッシュして高速化（おまじない）
  api.cache(true);
  return {
    // presets: [ ... ]（中身は次で解説）
  };
};
```

- `babel.config.js` は「**設定オブジェクトを返す関数**」を外に公開する形になっています。`function (api) { ... return { ... } }` の `return` した中身がBabelの設定です。
- `api.cache(true)` は「**この設定を毎回作り直さず、結果を覚えておく**」ための1行です。変換を速くするための**おまじない**で、ほぼ必ずこの形で書きます。
- `api` はBabelが渡してくる「設定用の道具箱」で、ここでは `cache` だけ使っています。

> **用語:** キャッシュ（cache） … 一度作った結果を覚えておき、次回はそれを使い回す仕組み。毎回同じ計算をやり直さずに済むので速くなります。

解説2: 変換ルールの一式（`presets`）

コピー

```
    presets: [
      ["babel-preset-expo", { jsxImportSource: "nativewind" }],
      "nativewind/babel",
    ],
```

- `presets` は「**どんな変換ルールを使うか**」のリストです。上から順に2つ指定しています。
- `["babel-preset-expo", { jsxImportSource: "nativewind" }]` … Expo標準の変換ルールに、オプション `{ jsxImportSource: "nativewind" }` を渡しています。これが「**`<View className="...">` のようなJSXを変換するとき、NativeWindの仕組みを使ってね**」という肝心の指定です。
- `"nativewind/babel"` … `className` を解釈してスタイルに変えるための、NativeWind専用の変換ルールです。
- つまりこの2つで、**Babel側に「classNameをスタイルへ翻訳するルール」を組み込んでいます**。基本は丸ごとコピーで構いません。

> **用語:** プリセット（preset） … 「よく使う変換ルールをひとまとめにしたセット」のこと。1つ指定するだけで、必要な変換がまとめて有効になります。

`metro.config.js`（無ければ作成）：

> **▼ このコードがやること（先に日本語で）**：Metro（ファイルを1つにまとめてスマホへ届ける道具）に「**NativeWindの機能を足し、先ほど作った `global.css` を読み込ませる**」設定です。
> `withNativeWind(...)` は「Expoの標準設定を受け取って、NativeWind対応版にして返す」関数だと考えてください。これも基本はコピーでOK。詳しい役割は下のblockquoteと各行のコメントにあります。

コピー

```
// metro.config.js — バンドラ(Metro)の設定。global.cssをNativeWindに渡す
const { getDefaultConfig } = require("expo/metro-config");
const { withNativeWind } = require("nativewind/metro");

// Expoの標準設定を取得
const config = getDefaultConfig(__dirname);

// withNativeWind : 標準設定にNativeWindの機能を足す。input に先ほどのCSSを指定
module.exports = withNativeWind(config, { input: "./app/global.css" });
```

2つの設定ファイルが何をしているか（要点）： - **`babel.config.js`** … `babel-preset-expo`（Expo標準の変換ルール）に、`{ jsxImportSource: "nativewind" }` を渡しています。これは「**JSX（`<View className="...">` のような画面の書き方）を変換するとき、NativeWindの仕組みを使ってね**」という指定です。続く `"nativewind/babel"` も、classNameを解釈するための変換ルールです。つまりBabel側に「classNameをスタイルに変える翻訳ルール」を追加しています。 - **`metro.config.js`** … `getDefaultConfig` でExpoの標準設定を取得し、`withNativeWind(config, { input: "./app/global.css" })` で「**標準設定にNativeWindの機能を足し、先ほど作った `global.css` を読み込ませる**」設定にしています。`withNativeWind` は「設定を受け取って、NativeWind対応版にして返す」関数だと考えてください。

難しく見えますが、やっていることは「**BabelとMetroの両方に、NativeWindの変換を組み込む**」だけです。丸ごとコピーで構いませんが、「classNameがスタイルに化けるのは、この2ファイルのおかげ」と覚えておくと、もしスタイルが効かないとき真っ先にここを見直せます。

▼ コードを1つずつ分解して解説

解説1: 必要な道具を読み込む（2つの `require`）

コピー

```
const { getDefaultConfig } = require("expo/metro-config");
const { withNativeWind } = require("nativewind/metro");
```

- ここでは、設定を組み立てるのに必要な**2つの関数を読み込んで**います。
- `getDefaultConfig` … Expoが用意している「**Metroの標準設定を取得する**」関数です。`expo/metro-config` から取り出しています。
- `withNativeWind` … 標準設定を受け取って「**NativeWind対応版に作り変えて返す**」関数です。`nativewind/metro` から取り出しています。
- `const { 名前 } = require(...)` は「読み込んだものの中から、必要な関数だけ名前で取り出す」書き方（分割代入）です。

用語: バンドラ（bundler） … たくさんのファイルや部品を1つにまとめてアプリへ届ける道具。React NativeではMetroがその役割を担います（第1章で登場）。

解説2: 標準設定を取得してNativeWind対応にする

コピー

```
// Expoの標準設定を取得
const config = getDefaultConfig(__dirname);

module.exports = withNativeWind(config, { input: "./app/global.css" });
```

- `getDefaultConfig(__dirname)` で、まず**Expoの標準設定**を `config` に受け取ります。`__dirname` は「この設定ファイルがある場所（フォルダのパス）」を表す決まり文句です。
- `withNativeWind(config, { input: "./app/global.css" })` で、その標準設定に**NativeWindの機能を足し**、先ほど作った `global.css` を読み込ませています。`input` に渡したCSSが、アプリ全体のスタイルの土台になります。
- それを `module.exports = ...` で外に公開することで、Metroがこの「NativeWind対応版の設定」を使うようになります。

> **用語:** `__dirname` …「今このファイルが置かれているフォルダの場所」を自動で表す特別な変数。設定ファイルでファイルの場所を伝えるときの定番です。

2.5 グローバルCSSを読み込む

`app/_layout.tsx`（第6章で作成）の先頭に、CSSの読み込みを1行足します。

コピー

```
// app/_layout.tsx の一番上に追加
// NativeWindのスタイルをアプリ全体で有効にする（先頭で読み込む）
import "./global.css";

import { Stack } from "expo-router";
//  （以下は第6章のまま）
```

2.6 設定を反映させる

設定ファイルを変えたときは、開発サーバーを**キャッシュを消して再起動**します。

コピー

```
npx expo start --clear
# --clear : 以前のキャッシュ（一時保存データ）を消して起動する。設定変更を確実に反映させる
```

> **`--clear` を付ける理由**: Metro（変換ツール）は速度のため変換結果を一時保存します。設定を変えた直後は古い保存が残っていて反映されないことがあるため、`--clear` で消してから起動します。「設定を変えたのに反映されない」ときの定番の対処です。

---

3. NativeWind の基本的な書き方

NativeWindでは、`style={...}` の代わりに `className="..."` にクラス名を並べて見た目を指定します。

> **▼ このコードがやること（先に日本語で）**: NativeWindの基本である **`className="..."` の書き方**を、画面中央に文字を表示する小さな例で体験します。ポイントは「`className` の中に、見た目を表す短いクラス名を**空白で区切って並べる**」こと——例えば `flex-1`（画面いっぱい）、`justify-center`（縦中央）、`text-2xl`（文字大きめ）のようにです。1つ1つのクラスが何を意味するかは、コード内のコメントで確認してください。

コピー

```
import { View, Text } from "react-native";

export default function Demo() {
  return (
    // className に空白区切りでクラスを並べる
    <View className="flex-1 justify-center items-center bg-white">
      {/* flex-1 → flex:1 / justify-center → 縦中央 / items-center → 横中央 / bg-white → 背景白 */}
      <Text className="text-2xl font-bold text-slate-800">📚 書籍管理アプリ</Text>
      {/* text-2xl → 文字大きめ / font-bold → 太字 / text-slate-800 → 濃いグレー文字 */}
    </View>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: 外側の枠を画面いっぱいに広げ、中身を中央へ（`<View>` のclassName）

コピー

```
<View className="flex-1 justify-center items-center bg-white">
```

- `className="..."` の中に、見た目を表す短いクラス名を**空白で区切って並べる**のがNativeWindの基本です。ここでは4つ並んでいます。
- `flex-1` … この `<View>` を「**使える空間いっぱいに広げる**」指定です（`flex: 1` と同じ）。今回は画面全体に広がります。
- `justify-center` … 中の要素を「**縦方向の中央**」にそろえます。

- `items-center` … 中の要素を「**横方向の中央**」にそろえます。この2つを合わせて「画面のど真ん中」に配置されます。
- `bg-white` … 背景を**白**にします（`bg-` は背景色の指定）。

> **用語:** Flexbox（フレックスボックス） … 要素の並べ方や中央寄せを決める仕組み。`flex-1`（広がる量）、`justify-center`（主軸方向の中央）、`items-center`（交差軸方向の中央）はこの仕組みのクラスです。

解説2: 文字の見た目を整える（`<Text>` のclassName）

コピー

```
<Text className="text-2xl font-bold text-slate-800">📚 書籍管理アプリ</Text>
```

- 文字を表示する `<Text>` にも `className` で見た目を当てます。ここでは3つ並んでいます。
- `text-2xl` … 文字を「**大きめのサイズ**」にします。`text-sm`（小）→ `text-base`（標準）→ `text-lg` / `text-xl` / `text-2xl`（大きく）と段階があります。
- `font-bold` … 文字を**太字**にします。
- `text-slate-800` … 文字色を「**濃いグレー**」にします。`text-色名-濃さ` の形で、濃さは50（薄い）〜900（濃い）で指定します。

> **用語:** カラースケール … Tailwind/NativeWindの色は「色名 + 濃さの数字（50〜900）」で表します。`slate-800` なら「スレート（青みグレー）の濃いほう」という意味です。

3.1 第4章のStyleSheetとの対応

第4章で書いたスタイルが、NativeWindのクラスにどう対応するかを表で示します。

やりたいこと	StyleSheet	NativeWind クラス
画面いっぱい	`flex: 1`	`flex-1`
縦方向中央	`justifyContent: "center"`	`justify-center`
横方向中央	`alignItems: "center"`	`items-center`
横並び	`flexDirection: "row"`	`flex-row`
内側余白16	`padding: 16`	`p-4`（4 = 16px）
文字サイズ大	`fontSize: 24`	`text-2xl`
太字	`fontWeight: "bold"`	`font-bold`
背景白	`backgroundColor: "#fff"`	`bg-white`
角丸	`borderRadius: 8`	`rounded-lg`
文字色青	`color: "#1e40af"`	`text-blue-800`

> **数字の対応（重要）**：Tailwindの余白やサイズの数字は「4倍するとピクセル」が基本です。`p-4` は `padding: 16`、`p-2` は `padding: 8`、`gap-3` は `gap: 12` です。慣れると暗算できます。色は `text-blue-600` のように「色名-濃さ（50〜900）」で指定します。

---

4. 一覧画面をNativeWindで書き直す

第7・8章で `StyleSheet` で書いた一覧画面を、NativeWindで書き直してみます。`StyleSheet.create` のブロックがごっそり消えて、コードが短くなるのを体感してください。

> **▼ このコードがやること（先に日本語で）**：これまで `StyleSheet` で書いていた一覧画面を、**すべて `className` で書き直した完成形**です。注目してほしいのは、ファイル末尾にあった `StyleSheet.create({ ... })` の長いブロックがまるごと消え、見た目の指定が各部品の `className` に移っている点です。検索ボックス・カード・追加ボタンなどの見た目が、短いクラス名だけで表現されています。各クラスの意味やテンプレートリテラルでの色切り替えは、コード内コメントと直後の補足を見てください。

コピー

```
// app/index.tsx — NativeWind版（主要部分のみ）

import { useState, useCallback, useMemo } from "react";
import { View, Text, FlatList, Pressable, TextInput, ActivityIndicator } from "react-native";
import { useRouter, useFocusEffect } from "expo-router";
import { fetchBooks } from "../lib/books";
import { Book } from "../types/book";

export default function HomeScreen() {
  const router = useRouter();
  const [books, setBooks] = useState<Book[]>([]);
  const [loading, setLoading] = useState(true);
  const [keyword, setKeyword] = useState("");

  const load = useCallback(async () => {
    try {
      setLoading(true);
      setBooks(await fetchBooks());
    } finally {
      setLoading(false);
    }
  }, []);

  useFocusEffect(useCallback(() => { load(); }, [load]));

  const filteredBooks = useMemo(() => {
    if (keyword.trim() === "") return books;
    const lower = keyword.toLowerCase();
    return books.filter(
      (b) => b.title.toLowerCase().includes(lower) || b.author.toLowerCase().includes(lower)
    );
  }, [books, keyword]);

  if (loading) {
    // className で中央寄せ。StyleSheet不要
    return (
      <View className="flex-1 justify-center items-center">
        <ActivityIndicator size="large" color="#1e40af" />
      </View>
    );
  }

  return (
    <View className="flex-1 bg-slate-50">
      {/* 検索ボックス */}
      <View className="p-4 pb-0">
        <TextInput
          className="bg-white border border-slate-200 rounded-lg px-4 py-2.5 text-sm"
          placeholder="🔍 タイトルや著者で検索..."
          value={keyword}
          onChangeText={setKeyword}
        />
      </View>

      <FlatList
        data={filteredBooks}
        keyExtractor={(item) => item.id}
        // contentContainerStyleのNativeWind版
        contentContainerClassName="p-4 gap-2.5"
        ListEmptyComponent={
          <Text className="text-center text-slate-400 mt-10 px-5">
            {keyword.trim() === "" ? "まだ本が登録されていません。" : "該当する本が見つかりません。"}
          </Text>
        }
        renderItem={({ item }) => (
          // カード。borderやrounded、paddingをすべてクラスで指定
          <Pressable
```

```
            className="bg-white rounded-xl p-3.5 border border-slate-200"
            onPress={() => router.push(`/books/${item.id}`)}
          >
            <Text className="text-base font-bold text-slate-800">{item.title}</Text>
            <Text className="text-xs text-slate-500 mt-0.5">著者: {item.author}</Text>
            {/* ステータスバッジ。色はstatusに応じてクラスを切り替える */}
            <View className={`self-start mt-2 px-2.5 py-0.5 rounded-full ${getStatusClass(item.status)}`}>
              <Text className="text-xs font-semibold text-slate-700">{item.status}</Text>
            </View>
          </Pressable>
        )}
      />

      {/* 追加ボタン（FAB） */}
      <Pressable
        className="absolute right-5 bottom-8 w-14 h-14 rounded-full bg-blue-800 justify-center items-center shadow-lg"
        onPress={() => router.push("/new")}
      >
        <Text className="text-white text-3xl leading-8">＋</Text>
      </Pressable>
    </View>
  );
}

// ステータスごとの背景色クラスを返す（第7章のgetStatusStyleのNativeWind版）
function getStatusClass(status: string) {
  if (status === "読了") return "bg-green-100";
  if (status === "読書中") return "bg-blue-100";
  return "bg-amber-100";
}
```

> **`className` の中で `${ }` を使う:** `className={`... ${getStatusClass(item.status)}`}` のように、テンプレートリテラルで動的にクラスを足せます。ステータスによってバッジの背景色を変える、といった「条件で見た目を変える」処理に便利です。
>
> **`contentContainerClassName`：** `FlatList` の内側のコンテナに対するクラス指定です。`StyleSheet` 版の `contentContainerStyle` に対応します。NativeWindは主要な部品でこうした `〜ClassName` を用意しています。
>
> **StyleSheetと混在してOK:** すべてを一度に書き換える必要はありません。NativeWindと `StyleSheet` は同じプロジェクトで共存できます。新しい画面はNativeWind、既存はそのまま、でも問題ありません。少しずつ慣れていきましょう。

▼ コードを1つずつ分解して解説

ここでは、見た目を決めている **`className` の塊**を中心に取り上げます。ロジック（`useState` や `useMemo` など）は第7・8章で解説済みなので、本節では「クラス名が何を意味するか」に絞って読み解きます。

解説1: 読み込み中の中央配置（ローディング画面）

コピー

```
<View className="flex-1 justify-center items-center">
  <ActivityIndicator size="large" color="#1e40af" />
</View>
```

- データ取得中（`loading` が `true`）のときに表示する、**くるくる回る読み込み表示**の枠です。
- `flex-1` … この `<View>` を画面いっぱいに広げます。
- `justify-center`（縦中央）と `items-center`（横中央）の組み合わせで、`ActivityIndicator`（読み込みアイコン）が**画面のど真ん中**に来ます。
- 見た目の指定が `className` の3語だけで済み、`StyleSheet.create` のブロックが不要になっている点に注目してください。

> **用語:** `ActivityIndicator` … React Native標準の「処理中を示すくるくるアイコン」。`size` で大きさ、`color` で色を指定します。

解説2: 一覧画面の外枠（背景色つきの全画面コンテナ）

コピー

```
<View className="flex-1 bg-slate-50">
```

- 一覧画面全体を包む一番外側の枠です。
- `flex-1` … 画面いっぱいに広げます。
- `bg-slate-50` … 背景を「**ごく薄いグレー**」にします。`50` は一番薄い濃さで、白に近い上品な背景色になり、上に乗る白いカードが少し浮いて見えます。

> **用語:** `bg-slate-50` … `bg-`（背景色）+ `slate`（青みのあるグレー）+ `50`（最も薄い濃さ）。背景をほんのり色づけたいときの定番です。

解説3: 検索ボックス（余白つき枠と入力欄のスタイル）

コピー

```
<View className="p-4 pb-0">
  <TextInput
    className="bg-white border border-slate-200 rounded-lg px-4 py-2.5 text-sm"
    placeholder="🔍 タイトルや著者で検索..."
    value={keyword}
    onChangeText={setKeyword}
  />
</View>
```

- 外側の `<View className="p-4 pb-0">` … `p-4` で**四方に16pxの余白**を付けつつ、`pb-0` で「**下の余白だけ0**」にしています（下にあるリストと間隔が空きすぎないようにする調整）。
- 入力欄 `<TextInput>` の `className` は5つの塊でできています。
- `bg-white` … 背景を白に。
- `border border-slate-200` …「**枠線あり**」（`border`）かつ「**枠線色は薄いグレー**」（`border-slate-200`）。
- `rounded-lg` … **角をやや丸く**します。
- `px-4 py-2.5` … **左右に16px・上下に10pxの内側余白**。`px` は横方向、`py` は縦方向の余白です。
- `text-sm` … 入力文字を**やや小さめ**に。

> **用語:** `px` / `py` / `pb` … `p`（padding＝内側の余白）に方向を足した書き方。`x` =左右、`y` =上下、`b` =下（`t` =上，`l` =左，`r` =右）。数字は基本「4倍するとpx」（`p-4` =16px、`py-2.5` =10px）です。

解説4: リスト全体の余白とすき間（`contentContainerClassName`）

コピー

```
<FlatList
  data={filteredBooks}
  keyExtractor={(item) => item.id}
  // contentContainerStyleのNativeWind版
  contentContainerClassName="p-4 gap-2.5"
  ...
```

- `contentContainerClassName` は「`FlatList` **の内側（中身を並べるコンテナ）に当てるクラス**」です。`StyleSheet` 版の `contentContainerStyle` に対応します。
- `p-4` … リストの中身の**四方に16pxの余白**を付け、画面端に貼り付かないようにします。
- `gap-2.5` … 並んだカード同士の**すき間を10px**空けます。`gap` を使うと、各カードに余白を付けなくても一定間隔で並びます。

> **用語:** `gap` … 並んだ要素同士の「あいだのすき間」をまとめて指定する書き方。1つ1つに `margin` を付けるより簡潔に等間隔を作れます。

解説5: 空のときのメッセージ（中央寄せ・薄色・上余白）

コピー

```
<Text className="text-center text-slate-400 mt-10 px-5">
  {keyword.trim() === "" ? "まだ本が登録されていません。" : "該当する本が見つかりません。"}
</Text>
```

- リストが空のとき（`ListEmptyComponent`）に表示する案内文です。
- `text-center` … 文字を**中央寄せ**にします。
- `text-slate-400` … 文字色を「**薄めのグレー**」にして、目立ちすぎない控えめな案内にします。
- `mt-10` … **上に40pxの余白**（`mt` = margin-top）を空け、画面の少し下に表示します。
- `px-5` … 左右に20pxの余白を付け、長文でも画面端に触れないようにします。

> **用語:** `mt` … `m`（margin＝外側の余白）の上方向（top）。要素の「外側」に間隔を空けたいときは `m` 系、「内側」に空けたいときは `p` 系を使います。

解説6: 本のカード（白背景・角丸・枠線）

コピー

```
<Pressable
  className="bg-white rounded-xl p-3.5 border border-slate-200"
  onPress={() => router.push(`/books/${item.id}`)}
>
  <Text className="text-base font-bold text-slate-800">{item.title}</Text>
  <Text className="text-xs text-slate-500 mt-0.5">著者: {item.author}</Text>
```

- 1冊分のカードを表す `<Pressable>`（押せる領域）のスタイルです。
- `bg-white` … カード背景を白に（薄グレー背景の上で浮いて見えます）。
- `rounded-xl` … **角を大きめに丸く**します（`lg` よりさらに丸い）。
- `p-3.5` … 四方に**14pxの内側余白**を付けます。
- `border border-slate-200` … 薄いグレーの**枠線**を付けて輪郭をはっきりさせます。
- カード内のテキスト。
- タイトルは `text-base`（標準サイズ）＋ `font-bold`（太字）＋ `text-slate-800`（濃いグレー）で見出しらしく。

- 著者名は `text-xs`（小さめ）＋ `text-slate-500`（中間グレー）＋ `mt-0.5`（上に2pxのわずかな余白）で、タイトルの下に控えめに添えます。

> **用語:** `<Pressable>` … タップに反応するReact Nativeの部品。`onPress` に押されたときの処理を渡します。ここでは押すと詳細画面へ移動します。

解説7: ステータスバッジ（テンプレートリテラルで色を切り替える）

コピー

```
<View className={`self-start mt-2 px-2.5 py-0.5 rounded-full ${getStatusClass(item.status)}`}>
  <Text className="text-xs font-semibold text-slate-700">{item.status}</Text>
</View>
```

- 「読了／読書中／未読」などの状態を示す**小さなバッジ**です。`className` がテンプレートリテラル（`` `...` ``）になっている点が新しいところです。
- `self-start` … バッジを「**左端に合わせ、横幅を中身ぴったり**」にします（横いっぱいに広がらないようにする指定）。
- `mt-2` … 上に8pxの余白。
- `px-2.5 py-0.5` … 左右10px・上下2pxの内側余白で、文字を小さな枠で囲みます。
- `rounded-full` … **角を完全に丸める**ことで、カプセル型（ピル型）のバッジになります。
- `${getStatusClass(item.status)}` … ここに**状態ごとの背景色クラス**が差し込まれます（次の解説8）。
- 中の文字は `text-xs`（小）＋ `font-semibold`（やや太字）＋ `text-slate-700`（やや濃いグレー）。

> **用語:** テンプレートリテラル … バッククォート `` ` `` で囲んだ文字列。中で `${ }` を使うと、変数や関数の戻り値を埋め込めます。固定のクラスに「条件で変わるクラス」を足すときに便利です。

解説8: 状態に応じて背景色クラスを返す関数（`getStatusClass`）

コピー

```
function getStatusClass(status: string) {
  if (status === "読了") return "bg-green-100";
  if (status === "読書中") return "bg-blue-100";
  return "bg-amber-100";
}
```

- バッジの**背景色クラスだけ**を、状態の文字列に応じて返す小さな関数です（第7章の `getStatusStyle` のNativeWind版）。

- `status` が `"読了"` なら `bg-green-100`（**薄い緑**）、`"読書中"` なら `bg-blue-100`（**薄い青**）を返します。
- どちらにも当てはまらない場合（未読など）は、最後の `return "bg-amber-100"`（**薄い黄色**）が使われます。
- いずれも濃さ `100` の**淡い背景色**なので、上に乗る濃いめの文字（`text-slate-700`）が読みやすくなります。

> **用語:** `bg-green-100` など … 背景色を「薄い色（濃さ100）」で当てるクラス。バッジやタグのように「うっすら色を付けて種類を区別したい」場面でよく使います。

解説9: 追加ボタン（画面右下に浮かぶ丸ボタン＝FAB）

コピー

```
<Pressable
  className="absolute right-5 bottom-8 w-14 h-14 rounded-full bg-blue-800 justify-center items-center shadow-lg"
  onPress={() => router.push("/new")}
>
  <Text className="text-white text-3xl leading-8">＋</Text>
</Pressable>
```

- 画面の右下に**常に浮いている丸い追加ボタン**（FAB＝Floating Action Button）のスタイルです。クラスが多いので塊ごとに見ます。
- `absolute right-5 bottom-8` … **位置を固定**（`absolute`）し、右から20px・下から32pxの位置に置きます。これでスクロールしても右下に浮き続けます。
- `w-14 h-14` … **幅と高さを56px**にして正方形に（`w`＝幅, `h`＝高さ）。
- `rounded-full` … 角を完全に丸めて**真円**にします。
- `bg-blue-800` … 背景を**濃い青**に。
- `justify-center items-center` … 中の「＋」を**上下左右の中央**に置きます。
- `shadow-lg` … **大きめの影**を付けて、ボタンが浮いて見えるようにします。
- 中の `<Text className="text-white text-3xl leading-8">` …「＋」を**白**（`text-white`）・**大きい文字**（`text-3xl`）にし、`leading-8`（行の高さ32px）で上下位置を微調整しています。

> **用語:** FAB（Floating Action Button） … 画面に固定で浮かぶ丸い操作ボタン。`absolute` で位置を固定し、`rounded-full` で円形にするのが定番の作り方です。

---

5. アプリ全体の仕上げ

機能と見た目が整ったら、公開前に「アプリらしさ」を高める仕上げをしておくと完成度が上がります。

5.1 アプリ名・アイコン・スプラッシュ画面

`app.json`（第1章で見た設定ファイル）で、アプリの基本情報を設定します。

> **▼ このコードがやること（先に日本語で）**：アプリの**名前・アイコン・起動画面（スプラッシュ）**などの基本情報を `app.json` にまとめて設定します。`name` はホーム画面に出るアプリ名、`icon` はアイコン画像、`splash` は起動時に一瞬出る画面の指定です。公開前の「アプリらしさ」を決める大事な設定なので、どの項目が何を表すかを各行のコメントで押さえておきましょう。

コピー

```
{
  "expo": {
    // ホーム画面に表示されるアプリ名
    "name": "書籍管理",
    // プロジェクトの識別名（半角英数）
    "slug": "my-books-app",
    // アプリのバージョン
    "version": "1.0.0",
    // アプリアイコン（1024x1024の正方形画像）
    "icon": "./assets/icon.png",
    // 起動時に一瞬出るスプラッシュ画面
    "splash": {
      // スプラッシュ画像
      "image": "./assets/splash.png",
      // その背景色
      "backgroundColor": "#1e40af"
    },
    // iPad対応
    "ios": { "supportsTablet": true },
    // Androidの識別子（後で変更）
    "android": { "package": "com.example.mybooksapp" }
  }
}
```

> **アイコンとスプラッシュ画像:** `assets` フォルダにExpoのデフォルト画像が入っています。まずはそのままでも公開できます。オリジナル画像に差し替えると一気に「自分のアプリ」感が出ます。アイコンは1024×1024ピクセルの正方形PNGを用意しましょう。第10章の公開前に整えるのがおすすめです。

▼ コードを1つずつ分解して解説

解説1: 設定全体の入れ物（`"expo"`）

コピー

```
{
  "expo": {
    "name": "書籍管理",
    "slug": "my-books-app",
    "version": "1.0.0"
  }
}
```

- `app.json` はExpoアプリの設定をまとめたファイルで、全体が `{ "expo": { ... } }` という形になっています。`"expo"` の中に各設定を並べます。
- `name` … **ホーム画面に表示されるアプリ名**です。日本語でもOK。
- `slug` … プロジェクトの**識別名**で、半角英数で付けます（URLなどに使われる内部用の名前）。
- `version` … アプリの**バージョン番号**。更新のたびに上げていきます。

> **用語:** JSON（ジェイソン） … `{ "キー": 値 }` の形で設定やデータを書く形式。値は文字列なら必ず `"..."` で囲み、項目はカンマ `,` で区切ります。

解説2: アイコンと起動画面（`icon` / `splash`）

コピー

```
    "icon": "./assets/icon.png",
    "splash": {
      "image": "./assets/splash.png",
      "backgroundColor": "#1e40af"
    }
```

- `icon` … ホーム画面に出る**アプリアイコンの画像ファイル**を指定します。1024×1024の正方形PNGが基本です。
- `splash` … アプリ起動時に**一瞬出る画面（スプラッシュ）**の設定をまとめたものです。
- `image` … スプラッシュに表示する画像。
- `backgroundColor` … その背景色。`#1e40af` は濃い青を表す**カラーコード**です。
- これらを設定すると、公開前の「自分のアプリらしさ」がぐっと高まります。

> **用語:** カラーコード（`#1e40af`） … 色を6桁の英数字で表す書き方。`#` のあとに赤・緑・青の強さを並べたもので、`#1e40af` は濃い青になります。

解説3: iOS・Androidごとの設定（`ios` / `android`）

コピー

```
    "ios": { "supportsTablet": true },
    "android": { "package": "com.example.mybooksapp" }
```

- `ios` と `android` は、それぞれの**OS専用の設定**をまとめる場所です。
- `ios.supportsTablet: true` … iPad（タブレット）に**対応する**という指定です。
- `android.package` … Androidアプリを**世界で1つに識別するための文字列**です。`com.会社名.アプリ名` のような形式で書き、公開前に自分用の値へ変更します。

> **用語:** パッケージ名（`com.example.mybooksapp`） … Androidでアプリを一意に識別するID。他のアプリと重複しないよう、自分が持つドメインを逆順にした形などで付けるのが慣習です。

5.2 ダークモード対応（発展・任意）

NativeWindは `dark:` というプレフィックスでダークモード（暗い配色）対応も簡単にできます。

コピー

```
// dark: を付けると、端末がダークモードのときだけそのクラスが適用される
<View className="bg-white dark:bg-slate-900">
  <Text className="text-slate-800 dark:text-slate-100">ダークモード対応のテキスト</Text>
</View>
```

> **任意の機能です:** ダークモード対応は必須ではありません。「こんなこともできる」という紹介です。余裕があれば挑戦してみてください。

---

6. この章のまとめ

- スタイリングの選択肢（StyleSheet / NativeWind / Tamagui / UIキット）を比較し、目的別の選び方を理解した
- **NativeWind** を導入（インストール → `tailwind.config.js` → `global.css` → Babel/Metro設定 → `--clear` 再起動）
- `style={...}` の代わりに `className="..."` でクラスを並べて見た目を指定する書き方を学んだ
- 一覧画面をNativeWindで書き直し、コードが短くなることを体感した
- `app.json` で**アプリ名・アイコン・スプラッシュ**を設定し、公開の準備を整えた

> **次の章へ:** アプリが完成しました！いよいよ第10章で、このアプリを **App Store と Google Play に公開**します。世界中の人があなたのアプリを使えるようになります。

発展: アプリでは使っていない重要なスタイリング機能

ここまでで作った書籍管理アプリでは使いませんでしたが、**実際のアプリ開発でとてもよく登場する**スタイリングの機能をまとめて紹介します。どれも「いつ使うか・なぜ必要か」が分かれば、難しくありません。本書では `StyleSheet`（React Native標準）で書きますが、NativeWindでもほぼ同じ考え方が使えます。

> **大事な前提（Webとの違い）**：React Nativeは**スマホの画面**に表示する仕組みで、ホームページ（Web）の見た目を作るCSSとは別物です。Webでよく使う `box-shadow: ...` という文字列や、マウスを乗せたときの `:hover`、ページに貼り付ける `position: fixed` などは**使えません**。React Nativeには専用の書き方が用意されているので、この節ではそれを学びます。

Flexboxの詳細（折り返し・基準サイズ・個別配置・伸び縮みの比率）

> **▼ このコードがやること（先に日本語で）**：要素を「横や縦にきれいに並べる」仕組みである **Flexbox** の、よく使う4つの機能を1つの画面で試します。具体的には、入りきらないときに**次の行へ折り返す**（ `flexWrap` ）、各要素の**基準の幅を決める**（ `flexBasis` ）、特定の1つだけ**配置をずらす**（ `alignSelf` ）、そして**残りの空間を何対何で分け合うか**（ `flex` の比率）です。タグの一覧や、画面を比率で分割するレイアウトを作るときに役立ちます。

コピー

```
import { View, Text, StyleSheet } from "react-native";

export default function FlexboxDemo() {
  return (
    <View style={styles.screen}>
      {/* (1) 折り返し: 横に並べて入りきらなければ次の行へ */}
      <View style={styles.tagRow}>
        <Text style={styles.tag}>React</Text>
        <Text style={styles.tag}>Native</Text>
        <Text style={styles.tag}>Flexbox</Text>
        <Text style={styles.tag}>StyleSheet</Text>
        <Text style={styles.tag}>NativeWind</Text>
      </View>

      {/* (2) 伸び縮みの比率: 1 : 2 で横幅を分け合う */}
      <View style={styles.ratioRow}>
        <View style={[styles.box, { flex: 1, backgroundColor: "#93c5fd" }]}>
          <Text>flex: 1</Text>
        </View>
        <View style={[styles.box, { flex: 2, backgroundColor: "#3b82f6" }]}>
          <Text>flex: 2</Text>
        </View>
      </View>

      {/* (3) 自分だけ右へ: alignSelf で1つだけ配置をずらす */}
      <View style={styles.selfRow}>
        <Text style={styles.selfItem}>左寄せ（親の指定どおり）</Text>
        <Text style={[styles.selfItem, { alignSelf: "flex-end" }]}>
          この行だけ右寄せ
        </Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, padding: 16, gap: 16 },
  tagRow: {
    // 横並びにする
    flexDirection: "row",
    // 入りきらなければ折り返す
    flexWrap: "wrap",
    // タグ同士のすき間
    gap: 8,
  },
  tag: {
    backgroundColor: "#e0e7ff",
    paddingHorizontal: 12,
```

```
    paddingVertical: 6,
    // 完全な丸み（カプセル型）
    borderRadius: 999,
    // 1つあたりの基準の幅を100にする
    flexBasis: 100,
    textAlign: "center",
  },
  ratioRow: { flexDirection: "row", gap: 8, height: 60 },
  box: { justifyContent: "center", alignItems: "center", borderRadius: 8 },
  selfRow: { gap: 8 },
  selfItem: { backgroundColor: "#fde68a", padding: 8, borderRadius: 8 },
});
```

▼ コードを1つずつ分解して解説

解説1: 折り返して並べる（`flexWrap`）

コピー

```
tagRow: {
  // 横並びにする
  flexDirection: "row",
  // 入りきらなければ折り返す
  flexWrap: "wrap",
  gap: 8,
},
```

- `flexDirection: "row"` で、中の要素を**横方向に並べます**（初期値は縦並びの `"column"`）。
- `flexWrap: "wrap"` は「**横幅が足りなくなったら、はみ出さずに次の行へ折り返す**」という指定です。これが無い（初期値の `"nowrap"`）と、要素が無理やり縮められたり画面からはみ出したりします。
- `gap: 8` で、並んだ要素同士のすき間を8空けます。折り返した後の行同士のすき間にも効きます。

> **用語: `flexWrap`（フレックスラップ）** … 並べた要素が**1行に収まらないとき折り返すかどうか**を決める指定。タグ一覧やボタンの集まりのように「数が変わるものを並べる」場面で必須です。

解説2: 1つあたりの基準サイズ（`flexBasis`）

コピー

```
tag: {
  // 1つあたりの基準の幅を100にする
  flexBasis: 100,
  // ...
},
```

- `flexBasis` は「**その要素の、最初の基準となる大きさ**」です。横並び（`row`）なら幅、縦並び（`column`）なら高さの基準になります。
- ここでは `100` を指定しているので、各タグは**まず幅100を目安に置かれ**、入りきらなければ前述の `flexWrap` で折り返されます。
- `width`（固定幅）と似ていますが、`flexBasis` は「**あくまで基準**であり、空き具合に応じて伸び縮みできる」点が違います。

> **用語:** `flexBasis`（フレックスベーシス） … 伸び縮みする前の「**スタートの大きさ**」。`width` がガチガチの固定なのに対し、こちらは柔軟に調整される基準値です。

解説3: 自分だけ配置をずらす（`alignSelf`）

コピー

```
<Text style={[styles.selfItem, { alignSelf: "flex-end" }]}>
  この行だけ右寄せ
</Text>
```

- `alignSelf` は「**親の配置ルールを無視して、自分1つだけ別の配置にする**」指定です。
- 親（`selfRow`）は縦並びなので、通常は子要素が左端からそろいます。そこに `alignSelf: "flex-end"` を付けた要素**だけ**が、右端（行の終わり側）に寄ります。
- `"flex-start"`（始め側）、`"center"`（中央）、`"flex-end"`（終わり側）、`"stretch"`（いっぱいに伸ばす）などが指定できます。

> **用語:** `alignSelf`（アラインセルフ） … 親がまとめて決めた配置を、**その要素だけ上書き**する指定。「基本は左寄せだけど、この1つだけ右に出したい」というときに使います。

解説4: 残りの空間を比率で分け合う（`flex` の数字）

コピー

```
<View style={[styles.box, { flex: 1, backgroundColor: "#93c5fd" }]}>
  <Text>flex: 1</Text>
</View>
<View style={[styles.box, { flex: 2, backgroundColor: "#3b82f6" }]}>
  <Text>flex: 2</Text>
</View>
```

- 横並びの中に置いた2つの枠に、それぞれ `flex: 1` と `flex: 2` を付けています。
- このとき2つは**幅を 1 : 2 の比率で分け合います**。つまり右側が左側の2倍の幅になります。`flex` の数字は「使える空間を、どんな割合で取り合うか」を表す**比率**です。
- 画面を「左に1・右に2」のように比率で分割したいとき、固定のピクセル数を計算しなくてもこれだけで実現できます。

> **用語:** `flex` の比率 … 並んだ要素に付けた `flex` の数字の**比**で、余った空間を分け合います。`flex: 1` と `flex: 1` なら半々、`flex: 1` と `flex: 3` なら 1 : 3 になります。

画面サイズに合わせる（`Dimensions` と `useWindowDimensions`）

> **▼ このコードがやること（先に日本語で）**：スマホには画面が大きい機種・小さい機種、さらに**縦持ち・横持ち**があります。この例では、今の**画面の幅・高さを取得**して、その値に応じて見た目を変えます（例: 幅が広い端末では文字を大きく）。これを使うと、どの端末でも崩れない「**端末に合わせて伸び縮みするレイアウト**」が作れます。
> `useWindowDimensions` は画面を回転したときも**自動で最新の値に更新される**のが便利なポイントです。

コピー

```
import { View, Text, StyleSheet, useWindowDimensions, Dimensions } from "react-native";

// 画面の今の大きさを「1回だけ」測る（更新はされない）
const initial = Dimensions.get("window");
console.log("起動時の幅:", initial.width);

export default function ResponsiveDemo() {
  // 画面の幅・高さを取得（回転や折りたたみで自動更新される）
  const { width, height } = useWindowDimensions();

  // 幅が400より広ければ「大きい画面」とみなす
  const isWide = width >= 400;

  return (
    <View style={styles.screen}>
      <Text style={{ fontSize: isWide ? 24 : 16 }}>
        画面の幅: {Math.round(width)} / 高さ: {Math.round(height)}
      </Text>
      {/* 画面幅の半分の幅を持つ四角を作る */}
      <View style={{ width: width / 2, height: 60, backgroundColor: "#3b82f6" }} />
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, padding: 16, gap: 16, justifyContent: "center" },
});
```

▼ コードを1つずつ分解して解説

解説1: 1回だけ測る `Dimensions.get`

コピー

```
const initial = Dimensions.get("window");
console.log("起動時の幅:", initial.width);
```

- `Dimensions.get("window")` は「**今の画面の大きさ（幅・高さ）を1回測って返す**」関数です。返ってきたものから `.width`（幅）や `.height`（高さ）を取り出せます。
- `"window"` は「アプリが使える画面の領域」を意味します（似たものに端末全体を表す `"screen"` もあります）。
- 注意点は、これは**測った瞬間の値で固定**されることです。画面を回転しても**自動では更新されません**。コンポーネントの外（毎回再計算しない場所）で、起動時の値を一度だけ知りたいときに向いています。

> **用語:** `Dimensions` （ディメンションズ） … 画面の大きさを測るためのReact Native標準の道具。`get("window")` でその時点の幅・高さが分かります。

解説2: 自動更新される `useWindowDimensions`

コピー

```
const { width, height } = useWindowDimensions();
const isWide = width >= 400;
```

- `useWindowDimensions()` は「**今の画面の幅・高さを返し、しかも画面が回転・サイズ変更されると自動で最新の値に更新してくれる**」便利な関数（フック）です。
- ここでは `{ width, height }` として幅と高さを取り出しています。回転して幅が変わると、この値も変わり、画面が**自動で再描画**されます。
- `isWide = width >= 400` のように、取得した幅をもとに「広い画面かどうか」を判定して、後の見た目を切り替えられます。

> **用語:** フック（hook） … `use〜` という名前で始まる、React/React Nativeの特別な関数。画面の状態や端末の情報を**自動で追いかけて**、変化したら画面を更新してくれます。

解説3: 取得した値で見た目を変える

コピー

```
<Text style={{ fontSize: isWide ? 24 : 16 }}>
  画面の幅: {Math.round(width)} / 高さ: {Math.round(height)}
</Text>
<View style={{ width: width / 2, height: 60, backgroundColor: "#3b82f6" }} />
```

- `fontSize: isWide ? 24 : 16` は「**広い画面なら文字を24、そうでなければ16にする**」という条件分岐です（`条件 ? 真のとき : 偽のとき`）。
- `width: width / 2` のように、取得した画面幅を**計算に使って**、画面幅のちょうど半分の四角を作っています。端末が変わってもいつでも「画面の半分」になります。
- `Math.round(...)` は小数点以下を四捨五入して、表示を見やすい整数にしています。

> **いつ使う？** 「タブレットでは2列、スマホでは1列」「横持ちのときだけ横並び」など、**端末や向きに応じてレイアウトを変えたい**ときに使います。リアルなアプリでは必須級の機能です。

影を付ける（iOSとAndroidで指定が違う）

▼ **このコードがやること（先に日本語で）**：カードやボタンを「**少し浮いて見える**」ようにする影の付け方です。ここが少しややこしいのですが、React Nativeでは **iPhone（iOS）とAndroidで影の指定方法が違います**。iOSは影の色・向き・濃さ・ぼかし具合を細かく指定し、Androidは `elevation`（高さ）という1つの数字で指定します。**両方を書いておけば**、どちらの端末でもきれいに影が付きます。

コピー

```
import { View, Text, StyleSheet } from "react-native";

export default function ShadowDemo() {
  return (
    <View style={styles.screen}>
      <View style={styles.card}>
        <Text>影付きのカード</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, padding: 24, backgroundColor: "#f8fafc", justifyContent: "center" },
  card: {
    backgroundColor: "#ffffff",
    padding: 24,
    borderRadius: 12,
    // ▼ iOS用の影（4つセットで指定する）
    // 影の色
    shadowColor: "#000000",
    // 影をどれだけずらすか（右0・下4）
    shadowOffset: { width: 0, height: 4 },
    // 影の濃さ（0=透明〜1=真っ黒）
    shadowOpacity: 0.15,
    // 影のぼかし具合（大きいほどふんわり）
    shadowRadius: 8,
    // ▼ Android用の影（高さを表す1つの数字）
    // 数字が大きいほど高く浮いて影が濃くなる
    elevation: 5,
  },
});
```

▼ コードを1つずつ分解して解説

解説1: iOS用の影は「4つセット」で指定する

コピー

```
// 影の色
shadowColor: "#000000",
// 影をどれだけずらすか
shadowOffset: { width: 0, height: 4 },
// 影の濃さ
shadowOpacity: 0.15,
// 影のぼかし具合
shadowRadius: 8,
```

- iPhone（iOS）では、影を**4つのプロパティの組み合わせ**で表現します。1つだけ書いても効かないので、基本は4つセットで指定します。
- `shadowColor` … 影の**色**です。たいてい黒（`"#000000"`）にします。
- `shadowOffset: { width: 0, height: 4 }` … 影を**どの向きにどれだけずらすか**。`height: 4` で「真下に4ずらした影」になり、要素が上に浮いて見えます。
- `shadowOpacity: 0.15` … 影の**濃さ**を `0`（透明）〜`1`（真っ黒）で指定します。`0.15` くらいが自然です。
- `shadowRadius: 8` … 影の**ぼかし具合**。大きいほど、ふわっと広がった柔らかい影になります。

> **用語:** 影の4点セット … iOSでは `shadowColor`（色）/ `shadowOffset`（ずらし）/ `shadowOpacity`（濃さ）/ `shadowRadius`（ぼかし）の4つで影を作ります。WebのCSSで使う `box-shadow` という1行の文字列は**使えない**ので注意してください。

解説2: Android用の影は「高さ」1つで指定する

コピー

```
// 数字が大きいほど高く浮いて影が濃くなる
elevation: 5,
```

- Androidでは影を `elevation`（エレベーション＝高さ）という**1つの数字**だけで指定します。iOSの4点セットは効きません。
- 数字が大きいほど「**より高く浮いている**」とみなされ、影が濃く大きくなります。`5` 前後がカードらしい自然な見た目です。
- 大事なのは、**iOS用の4点セットとAndroidの `elevation` の両方を書いておく**ことです。React Nativeは「その端末で有効なほうだけ」を使うので、両方書いておけばどちらの機種でも影が付きます。

> **いつ使う？** カード・ボタン・メニューなどを「**背景から少し浮かせて目立たせたい**」ときに使います。前の節で見たFAB（浮かぶ追加ボタン）も、影が付くことで「押せそうな浮いたボタン」に見えます。

変形させる（拡大・回転・移動＝ `transform` ）

> **▼ このコードがやること（先に日本語で）**：要素を**大きくしたり（拡大）、傾けたり（回転）、位置をずらしたり（移動）**する `transform` の使い方です。元のレイアウトを崩さずに「見た目だけ」変えられるのが特徴で、アイコンを少し傾けたり、押したとき一瞬大きくする演出などに使います。`transform` は**変形の指定を配列（リスト）で複数並べる**点がポイントです。

コピー

```
import { View, Text, StyleSheet } from "react-native";

export default function TransformDemo() {
  return (
    <View style={styles.screen}>
      {/* 1.3倍に拡大 */}
      <View style={[styles.box, { transform: [{ scale: 1.3 }] }]}>
        <Text>拡大</Text>
      </View>

      {/* 15度かたむける */}
      <View style={[styles.box, { transform: [{ rotate: "15deg" }] }]}>
        <Text>回転</Text>
      </View>

      {/* 右へ40ずらす + 少し縮小（複数を配列で並べる） */}
      <View
        style={[styles.box, { transform: [{ translateX: 40 }, { scale: 0.8 }] }]}
      >
        <Text>移動＋縮小</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, padding: 24, gap: 32, justifyContent: "center" },
  box: {
    backgroundColor: "#a7f3d0",
    padding: 16,
    borderRadius: 8,
    alignItems: "center",
  },
});
```

▼ コードを1つずつ分解して解説

解説1: 大きさを変える（`scale`）

コピー

```
transform: [{ scale: 1.3 }]
```

- `transform` には、変形の指定を **`[ { ... } ]` という配列（リスト）**で渡します。たとえ1つだけでも配列で囲みます。
- `{ scale: 1.3 }` は「**1.3倍に拡大する**」指定です。`1` が元のサイズ、`0.5` なら半分、`2` なら2倍です。
- 拡大しても**まわりのレイアウトの計算は元のサイズのまま**なので、近くの要素を押しのけずに「見た目だけ」大きくなります。

> **用語:** `scale`（スケール） … 拡大・縮小の倍率。`1` が等倍で、数字を大きくすると拡大、小さくすると縮小します。

解説2: 回転させる（`rotate`）

コピー

```
transform: [{ rotate: "15deg" }]
```

- `{ rotate: "15deg" }` は「**15度かたむける**」指定です。角度は `"15deg"` のように**文字列で単位 `deg`（度）を付けて**書きます。
- プラスの数字で時計回り、マイナス（`"-15deg"`）で反時計回りに回転します。
- アイコンを少し傾けたり、開閉する矢印を回したり、といった演出に使います。

> **用語:** `rotate`（ローテート） … 要素を回す指定。値は `"45deg"` のように**度数を文字列**で渡します（数字だけだとエラーになります）。

解説3: 位置をずらす＋複数を組み合わせる（`translateX` と配列）

コピー

```
transform: [{ translateX: 40 }, { scale: 0.8 }]
```

- `{ translateX: 40 }` は「**横方向（X）に40ずらす**」指定です（プラスで右、マイナスで左）。縦にずらす `translateY` もあります。
- ここでは配列の中に **`translateX` と `scale` の2つを並べて**います。このように `transform` は**複数の変形を組み合わせられる**のが大きな特徴です（「右に40ずらして、かつ0.8倍に縮小」）。
- 並べる順番にも意味があるので、思った見た目にならないときは順番を入れ替えてみてください。

いつ使う？ ボタンを押した瞬間に少し縮める、お知らせバッジをちょっと傾ける、要素をスッと横移動させるなど、「**見た目だけの小さな演出**」に使います。レイアウト自体は変えずに動きを足せるのが利点です。

要素を重ねて配置する（`position: "absolute"` と `zIndex`）

▼ このコードがやること（先に日本語で）：ふだん要素は順番に並びますが、`position: "absolute"`（絶対配置）を使うと、**親の中の好きな位置にピンで留めるように置ける**ようになります。さらに要素同士が重なったとき、`zIndex`（重なりの順番）で**どちらを前に出すか**を決められます。画像の右上に「NEW」バッジを重ねる、写真の上に文字を乗せる、といった表現に使います。

コピー

```
import { View, Text, StyleSheet } from "react-native";

export default function AbsoluteDemo() {
  return (
    <View style={styles.screen}>
      {/* この枠を「基準（relative）」にして、中の絶対配置がこの枠を基準に動く */}
      <View style={styles.card}>
        <Text>商品画像のかわりの枠</Text>

        {/* 右上に重ねるNEWバッジ */}
        <View style={styles.badge}>
          <Text style={styles.badgeText}>NEW</Text>
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, padding: 24, justifyContent: "center" },
  card: {
    // 中の絶対配置の「基準」になる
    position: "relative",
    height: 160,
    backgroundColor: "#e2e8f0",
    borderRadius: 12,
    justifyContent: "center",
    alignItems: "center",
  },
  badge: {
    // 基準の枠の中で位置を自由に指定
    position: "absolute",
    // 上から8
    top: 8,
    // 右から8
    right: 8,
    // 重なったとき手前に出す
    zIndex: 10,
    backgroundColor: "#ef4444",
    paddingHorizontal: 8,
    paddingVertical: 2,
    borderRadius: 999,
  },
  badgeText: { color: "#ffffff", fontSize: 12, fontWeight: "bold" },
});
```

▼ コードを1つずつ分解して解説

解説1: 基準になる枠を作る（`position: "relative"`）

コピー

```
card: {
  // 中の絶対配置の「基準」になる
  position: "relative",
  height: 160,
  // ...
},
```

- `position: "relative"` を付けた枠は、**中に置く絶対配置の要素の「基準」**になります。
- 絶対配置の要素（次の `badge`）は、`top` や `right` を「**一番近い `relative` の親からの距離**」として計算します。つまりこの枠を基準に、バッジの位置が決まります。
- これを付けておかないと、バッジが**画面全体を基準**にしてしまい、思わぬ位置に飛んでいくことがあります。

> **用語: `position: "relative"`** … その要素自体は普通に並ぶが、**中の絶対配置の「位置の基準点」になる**指定。「ここを基準にしてね」という土台の役割です。

解説2: 好きな位置にピン留めする（`position: "absolute"` と `top` / `right`）

コピー

```
badge: {
  // 基準の枠の中で位置を自由に指定
  position: "absolute",
  // 上から8
  top: 8,
  // 右から8
  right: 8,
  // ...
},
```

- `position: "absolute"` は「**通常の並びから外れて、指定した位置にピンで留める**」配置です。
- `top: 8` / `right: 8` で「基準の枠の**上から8・右から8**の位置」に置いています。これでカードの右上にバッジが乗ります。`top` / `bottom` / `left` / `right` の4方向で位置を指定できます。
- 絶対配置にした要素は「場所を取らない（まわりを押しのけない）」ので、他の要素の**上に重ねて**置けます。

用語: `position: "absolute"`（アブソリュート） … 要素を通常の並びから外し、`top` / `left` などで**好きな位置に固定**する配置。バッジ・ラベル・重ねる文字などに使います。なお、Webの「スクロールしても画面に貼り付く `fixed` 」はReact Nativeには無く、固定したいときはこの `absolute` を使います。

解説3: 重なりの前後を決める（`zIndex`）

コピー

```
// 重なったとき手前に出す
zIndex: 10,
```

- `zIndex` は「**要素が重なったとき、どちらを前（手前）に出すか**」を決める数字です。
- 数字が**大きいほど手前**に表示されます。`zIndex: 10` を付けたバッジは、`zIndex` の小さい（または無い）他の要素より前面に出ます。
- バッジや吹き出しが他の要素の下に隠れてしまうときは、この数字を上げると手前に出てきます。

いつ使う？ 商品画像の角に「SALE」「NEW」を重ねる、アイコンに未読数の赤丸を重ねる、写真の上に説明文を乗せる、といった「**要素を重ねて表現したい**」場面で使います。

OSごとにスタイルを変える（`Platform.select`）

▼ このコードがやること（先に日本語で）：同じアプリでも、iPhone（iOS）とAndroidでは「ちょうどいい見た目」が少し違うことがあります（影の付け方、フォント、余白など）。`Platform.select` を使うと、**今動いているOSに応じて自動で別々の値を使い分け**られます。ここでは、影の指定をiOSとAndroidで切り替える例を見ます。

コピー

```
import { View, Text, StyleSheet, Platform } from "react-native";

export default function PlatformDemo() {
  return (
    <View style={styles.screen}>
      <View style={styles.card}>
        {/* 今動いているOSの名前を表示 */}
        <Text>このOSは: {Platform.OS}</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, padding: 24, justifyContent: "center" },
  card: {
    backgroundColor: "#ffffff",
    padding: 24,
    borderRadius: 12,
    // OSごとに別々のスタイルを使い分ける
    ...Platform.select({
      ios: {
        // iOSのときだけ使う影の指定
        shadowColor: "#000000",
        shadowOffset: { width: 0, height: 4 },
        shadowOpacity: 0.15,
        shadowRadius: 8,
      },
      android: {
        // Androidのときだけ使う影の指定
        elevation: 5,
      },
    }),
  },
});
```

▼ コードを1つずつ分解して解説

解説1: 今のOSを知る（`Platform.OS`）

コピー

```
<Text>このOSは: {Platform.OS}</Text>
```

- `Platform` はReact Native標準の「**今どのOSで動いているかを教えてくれる道具**」です。

- `Platform.OS` には、iPhoneなら `"ios"`、Androidなら `"android"` という文字列が入っています。
- これを使えば「`if (Platform.OS === "ios") { ... }`」のように、OSによって処理を分けることもできます。

> **用語:** `Platform.OS` … 今動いているOSの名前（`"ios"` か `"android"`）が入った値。OSごとに違う動きをさせたいときの出発点です。

解説2: OSごとに値を選ぶ（`Platform.select`）

コピー

```
...Platform.select({
  ios: { /* iOSのときの影 */ },
  android: { elevation: 5 },
}),
```

- `Platform.select({ ios: ..., android: ... })` は「**今のOSに合うほうの中身を返す**」関数です。iOSで動いていれば `ios:` の中身、Androidなら `android:` の中身が選ばれます。
- 前の冒頭の `...`（スプレッド構文）は「**選ばれたオブジェクトの中身を、ここに展開して混ぜ込む**」書き方です。これにより、選ばれた影の指定が `card` のスタイルに合流します。
- 結果として、**iOSでは4点セットの影、Androidでは `elevation` の影**が自動で使い分けられます。前の「影」の節では両方ベタ書きしましたが、`Platform.select` を使うと「**そのOSに必要な分だけ**」を渡せて、すっきり書けます。

> **いつ使う？**「iOSとAndroidで影・フォント・余白の見た目をそろえたい」「片方のOSだけ特別な調整をしたい」ときに使います。OSごとの細かな違いを吸収する定番の道具です。
>
> **用語:** スプレッド構文（`...`） … オブジェクトや配列の**中身をその場に展開して並べる**書き方。ここでは「OSに応じて選ばれた設定を、スタイルの中に溶け込ませる」役割をしています。

長い文字を「…」で省略する（`numberOfLines` と `ellipsizeMode`）

> **▼ このコードがやること（先に日本語で）：**本のタイトルや説明文がとても長いと、画面からはみ出したりレイアウトが崩れたりします。`Text` に `numberOfLines`（表示する行数の上限）を付けると、**指定した行数を超えた分は自動で「…」に省略**されます。一覧のカードで「タイトルは1行まで」「説明は2行まで」とそろえたいときに大活躍する機能です。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
import { View, Text, StyleSheet } from "react-native";

export default function EllipsisDemo() {
  const longTitle = "とても長い本のタイトルが入っていて画面からはみ出してしまうケース";

  return (
    <View style={styles.screen}>
      {/* 1行を超えたら末尾を「…」で省略 */}
      <Text style={styles.title} numberOfLines={1} ellipsizeMode="tail">
        {longTitle}
      </Text>

      {/* 2行までにそろえる（説明文などに）*/}
      <Text style={styles.body} numberOfLines={2}>
        {longTitle}（こちらは2行まで表示し、3行目以降は省略されます）{longTitle}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, padding: 24, gap: 12, justifyContent: "center" },
  title: { fontSize: 18, fontWeight: "bold" },
  body: { fontSize: 14, color: "#475569" },
});
```

▼ コードを1つずつ分解して解説

解説1: 表示する行数の上限を決める（`numberOfLines`）

コピー

```
<Text style={styles.title} numberOfLines={1} ellipsizeMode="tail">
  {longTitle}
</Text>
```

- `numberOfLines={1}` は「**この文字は最大1行まで表示する**」という指定です。1行に収まらない長い文字は、はみ出さずに自動で切り詰められます。
- 切り詰められた箇所には「…（三点リーダー）」が表示され、「まだ続きがある」ことが分かります。
- `numberOfLines={2}` のように数字を変えれば「2行まで」「3行まで」と調整できます。カード内のタイトルや説明文の高さをそろえるのに最適です。

> **用語: `numberOfLines`（ナンバーオブラインズ）** … `Text` に「**最大で何行まで表示するか**」を指定するプロパティ。あふれた分は自動で「…」に省略されます。

解説2: どこを省略するか決める（`ellipsizeMode`）

コピー

```
ellipsizeMode="tail"
```

- `ellipsizeMode` は「**文字のどの部分を「…」にするか**」を決める指定です。
- `"tail"`（末尾）… いちばんよく使う指定で、**後ろを省略**します（`長いタイトル…`）。
- `"head"`（先頭）… **前を省略**します（`…いタイトル`）。`"middle"`（中央）… **真ん中を省略**します（`長い…トル`）。ファイル名のように末尾も見せたいときに便利です。
- 省略しても、`numberOfLines` を付けていれば自動で `"tail"` が初期値になるので、特にこだわりが無ければ省略してもかまいません。

> **いつ使う？** 一覧画面で「タイトルは1行・説明は2行」のように**カードの高さをそろえたい**とき、長いテキストでレイアウトが崩れるのを防ぎたいときに使います。実用アプリではほぼ必ず登場する機能です。